Singleton Design Pattern

Agenda
1. Intro to design patterns
2. Types of design patterns → Creational, Structural, Behavioural
3. Creational Design Patterns
   - Singleton design Pattern

What are design patterns?

- Something that keeps on repeating
- Software design

design pattern are nothing but well established solution to commonly occuring software design problem

→ well established sol^n to commonly occurring s/w design problems.

GoF ⇒ Gang of four book ≈ 23 design patterns

↳ Design Pattern

we will be covering 10 most important design pattern

≈ 10 design patterns

⇒
- importance in interviews
- commonly used in day to day work

Why learn DP ?

- → Shared vocabulary (vocabalary)
- → Save a lot of time
  - ↳ By not req you to do hit and trial
  - ↳ By not putting you into a mess.
- → Interviews

Types of design patterns

- ↳ Object oriented design.
- ↳ Sol to commonly occurring problems in OOPS

1. Creational design Patterns
   - → commonly occurring problems wrt creation of objects of an entity.
2. Structural design Pattern
   - → how should diff classes relate to each other
   - → what attrs/methods in a class.
   - → how your codebase should be structured

Student {

Batch {

}

}

(3) Behavioral Design Patterns

→ problems / intricacies wrt behaviours supported by a S/W system.

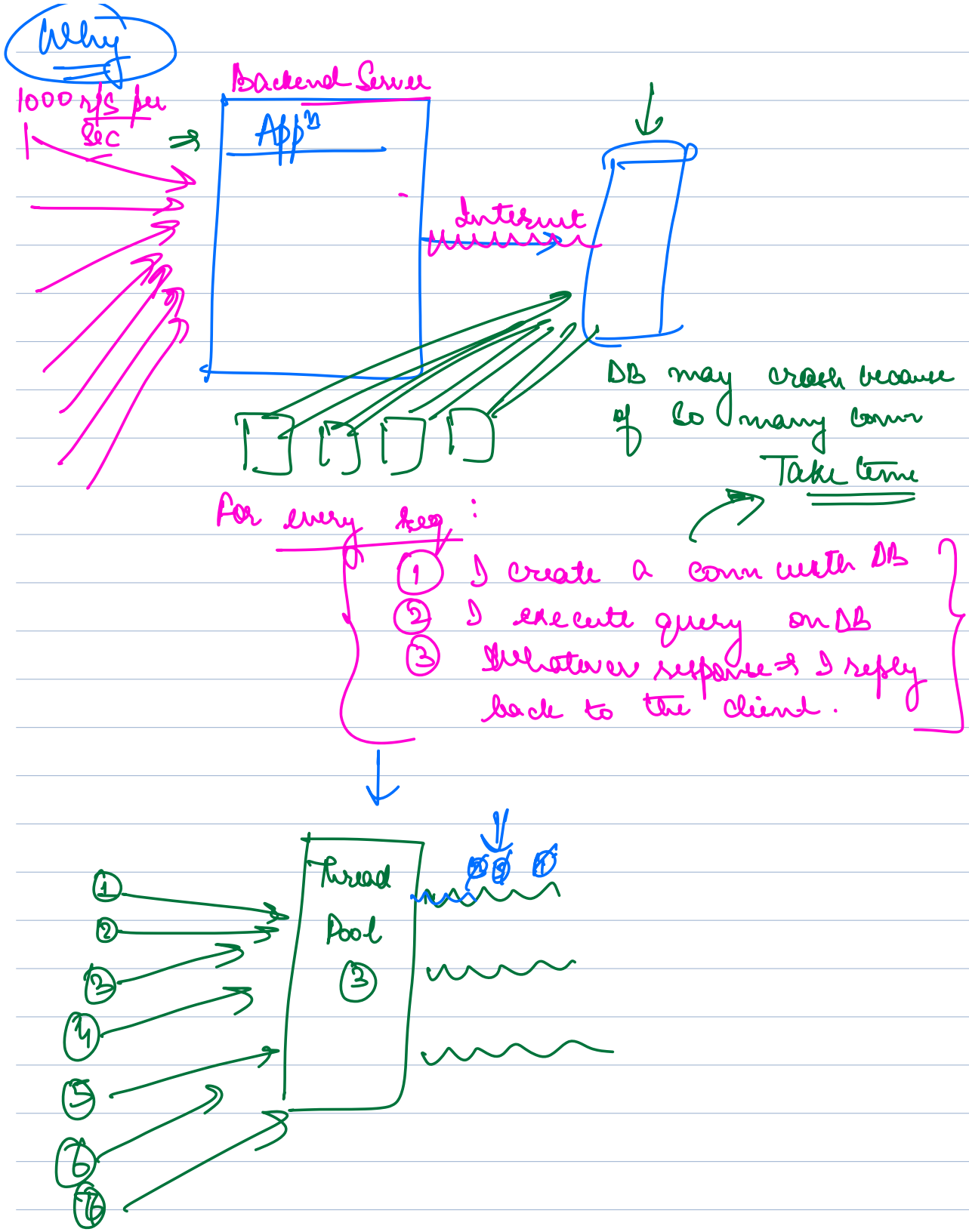
Just an intro! We will see a lot more examples of each in coming classes

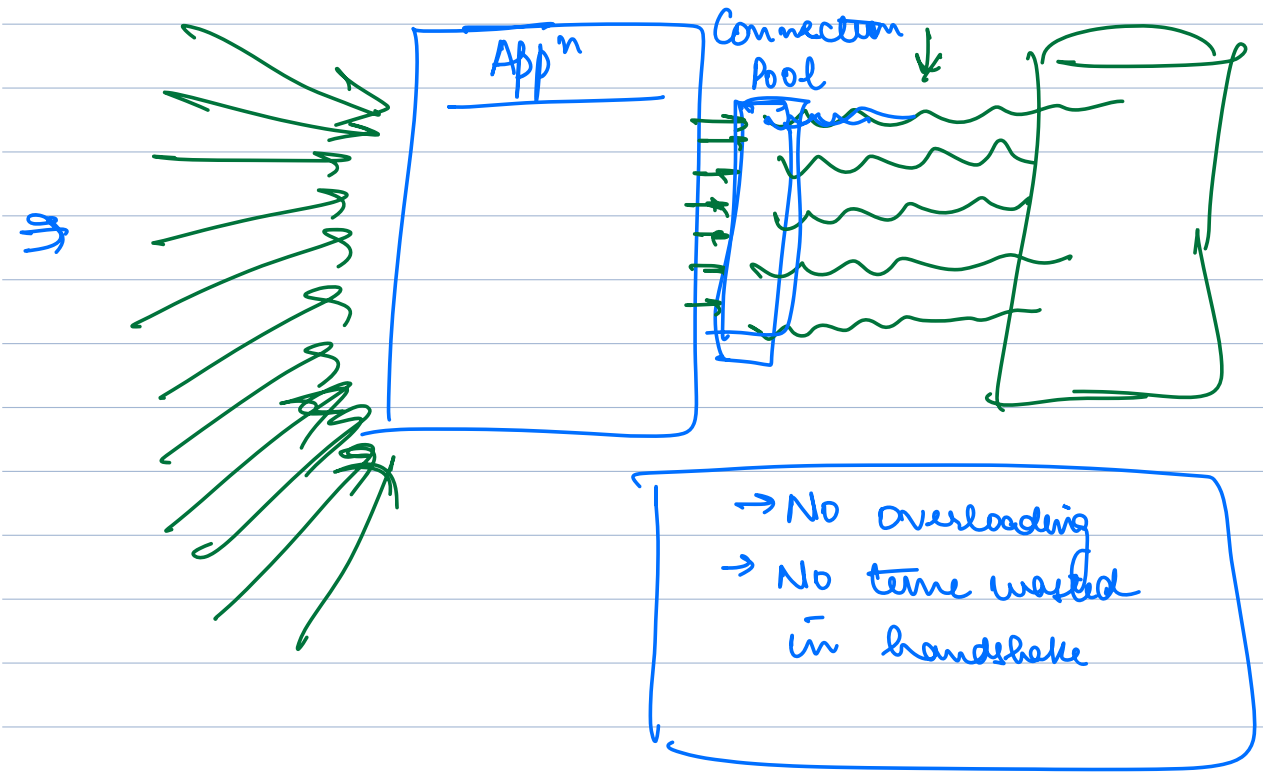
CREATIONAL DESIGN PATTERNS

- → Singleton
- → Builder
- → Factory
  - → Practical factory
  - → Factory Method
  - → Abstract Factory
- → Prototype

Singleton Design Pattern

Problem Statement → Why ?
→ How ?
→ What ?


Why
1000 req per sec
Backend Server
App
Internet
DB may crash because of too many conn
Take time
For every req :
1 I create a conn with DB
2 I execute query on DB
3 Whatever response → I reply back to the client.
Thread Pool
1
2
3
4
5
6
7




```
class Database {
    List<Connection> connPool;

    connect() {

    }
    executeQuery() {

    }

    disConnect() {

    }
}
```

bookmyshow

controllers

User Controller → Database

Booking Controller

Movie Controller. → Database

/users

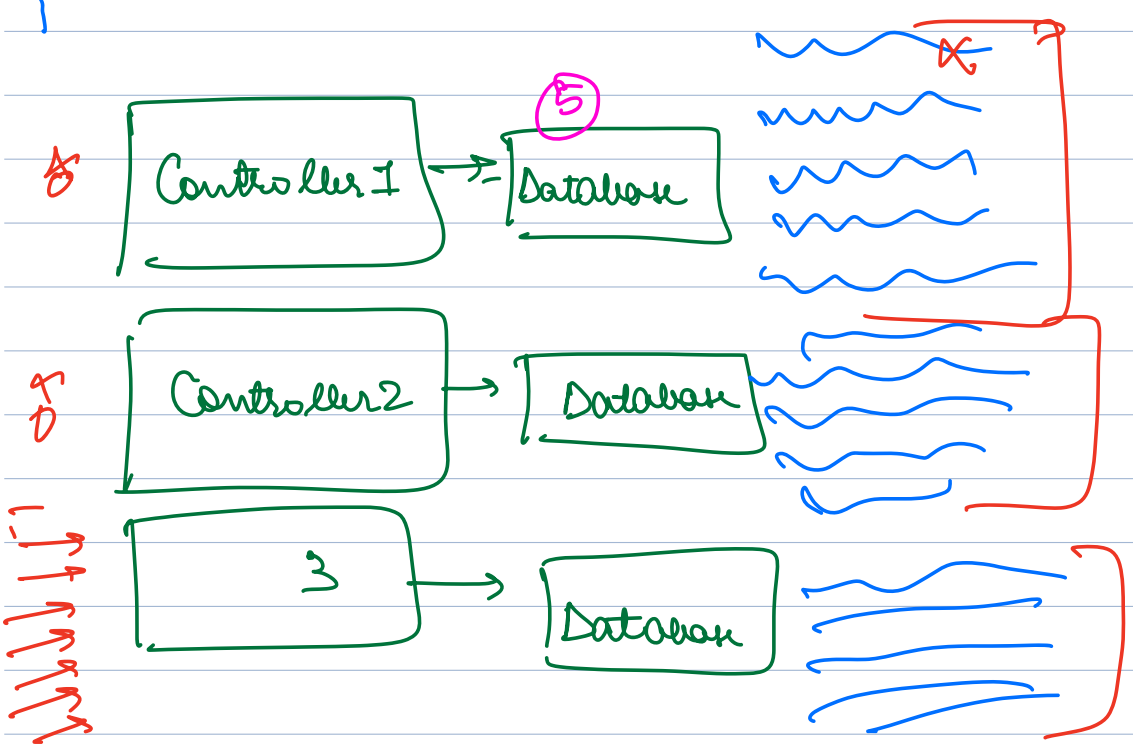
/book

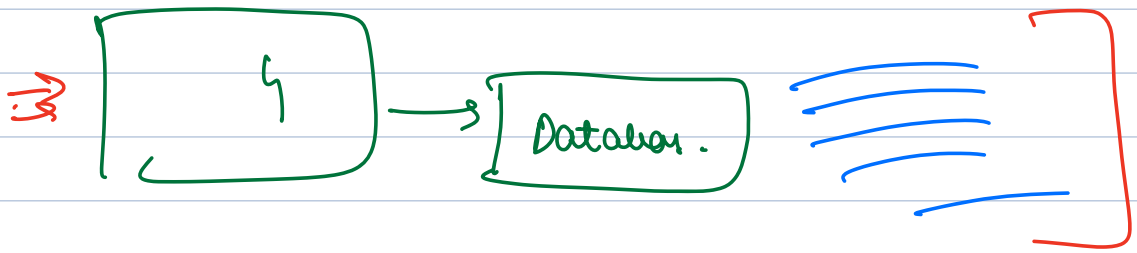
/movies

```
UserController {
    Database
    void login() {
    }
    void logout() {
    }
}
```

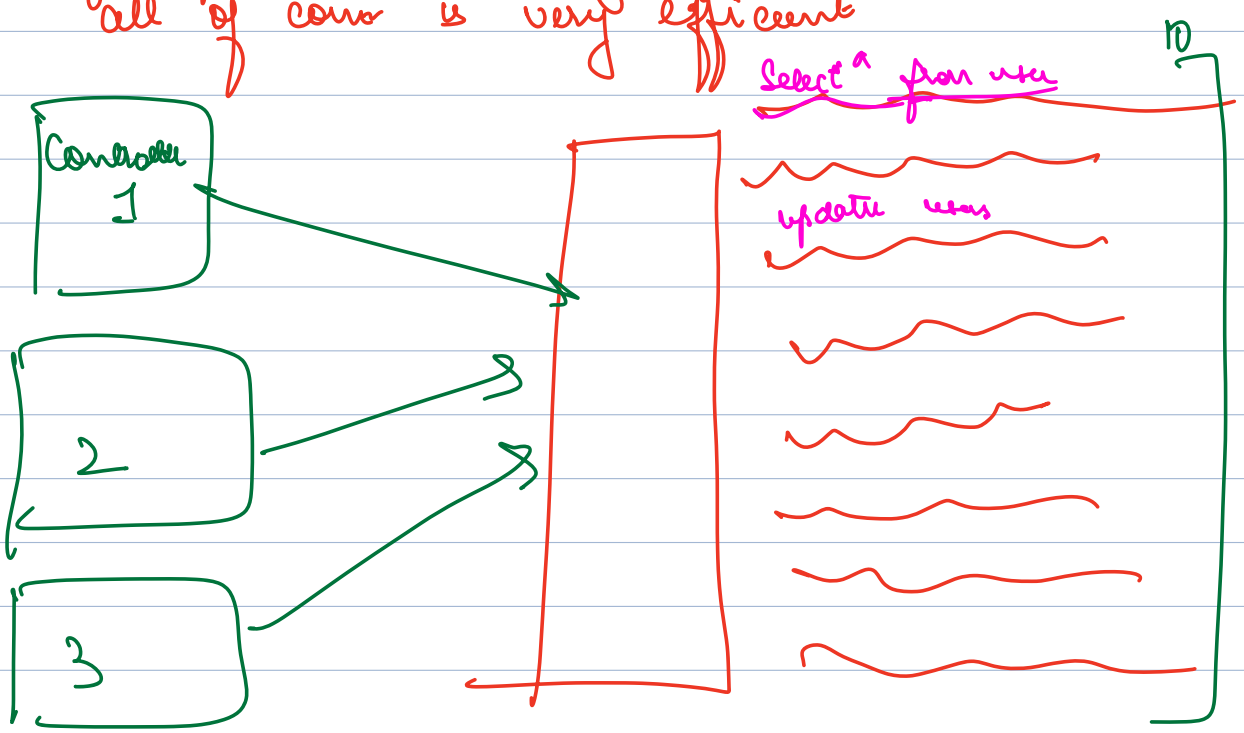
→ /users/login
→ /users/logout
→ /users/changepass

→ Every req is going to be served via one of the methods of one of the Controller.





Single place across my system that maintains all of conn is very efficient



→ Need a way to have only one object of conn pool across my system

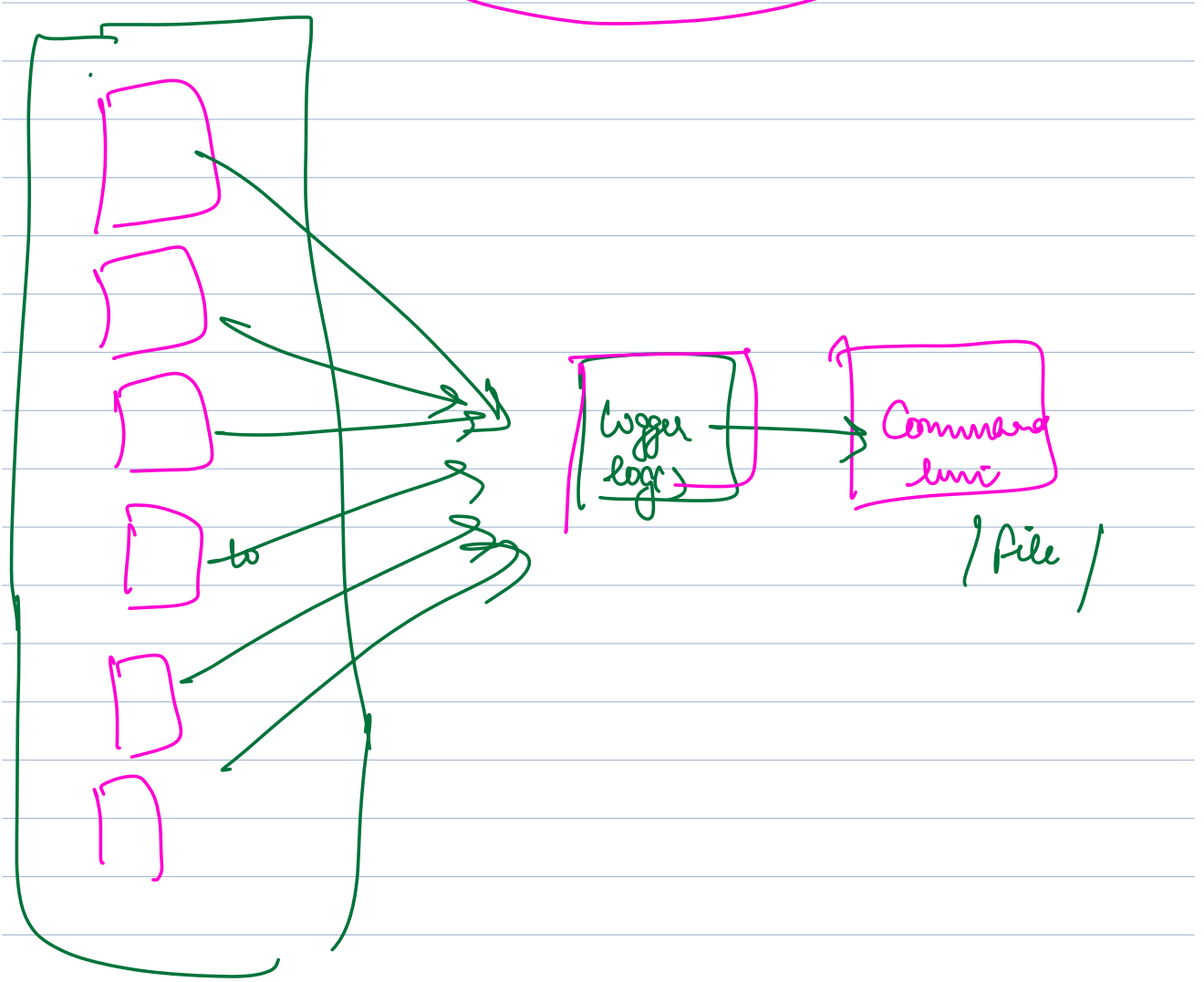
connection

any

Logging

→ CCTV for codebase

→ way to print something which allows to traceback what happened in the system in case of an error.

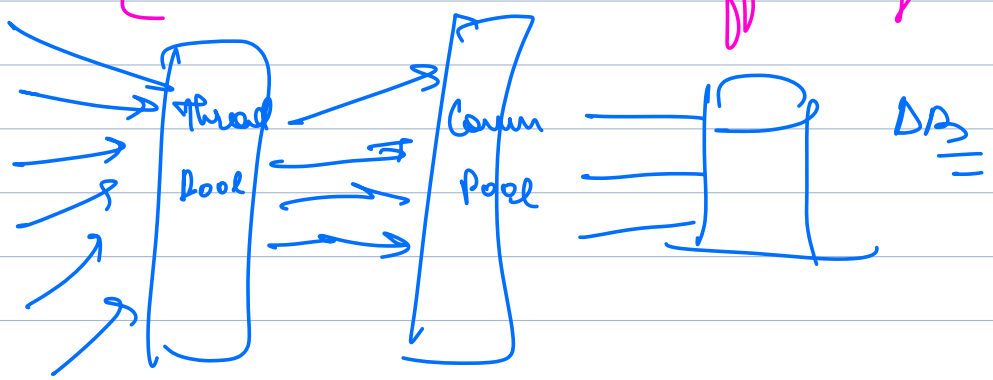


⇒ One obj can be used by multiple threads at same time

Q: ~~so if~~ one object cna be used by multiple threads at same time then how it is going to identiy that which object of particualr class will have particualr data

(1) A class which is having a shared resource required by many classes By

⇒ Single obj can manage how to use shared resource efficiently

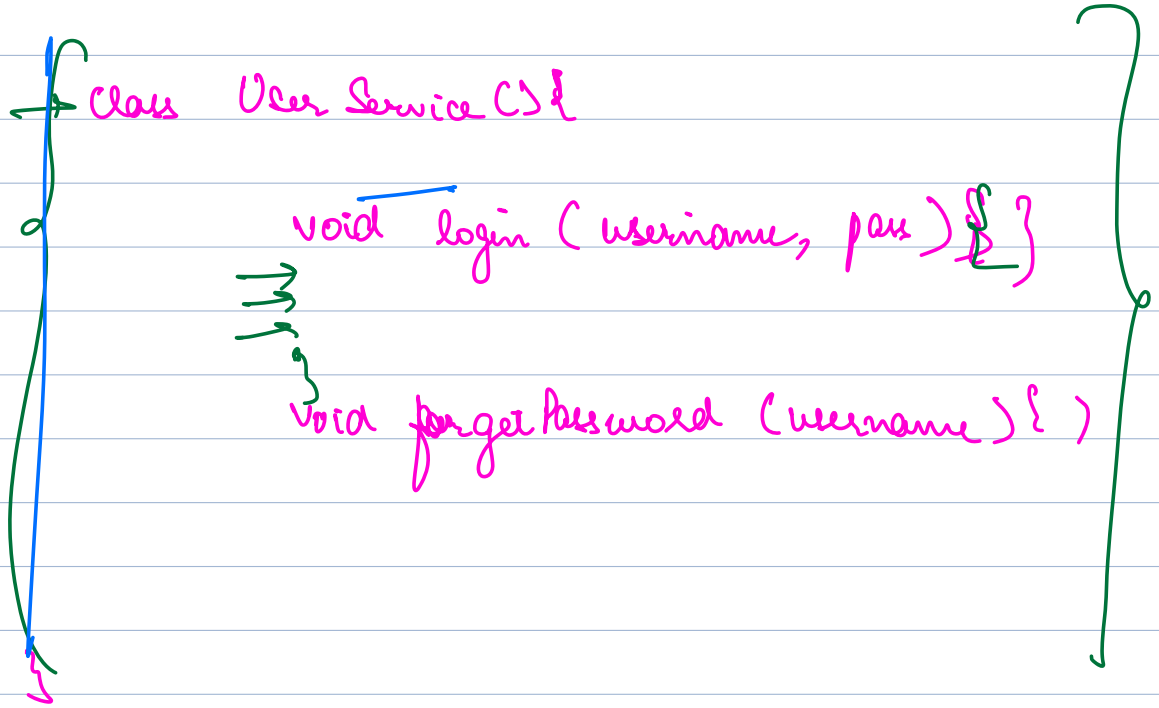


(2) Creation of an obj is not efficient, its expensive. Beacuse of we have to do belows

⇒ i/o call
⇒ tcp conn
⇒ Authorization
⇒ authentication

③ An immutable obj | An obj that has no fields.

↳ whose values you can't chg.



UserService us1 = new UserService()

us2 = _____

us1. login (_____)

us2. login( ) ⇒ us1. login( )

us1. login( )

Why Singletons?

1. Shared Resource → DB, Logger
2. No attr in obj / immutable → User Service
3. Obj creation is expensive → DB

⇒ we may want to have a class for which we have only 1 obj across my app

---

Singleton Design Pattern

How to create an obj that has only 1 instance across the app

How to implement Singleton

```
Class Database {

    String url;
    String username;
    String password;
    List<Connection> pool;

}
```

Goal: Make Database class singleton ⇒ only one obj of that can be created.

```
Class Database {
    String url;
    String username;
    String password;
    List<Connection> pool;
}
```

```
Database db1 = new DBC()
db2 = new DBC()
```

⇒ Till the time a class has a public constructor it can never be singleton.

↳ try making it private.

```
Class Database{
    =
    =
    =
    private Database(){}
}
```

→ But here I can't even create a single obj ⇒ not singleton

Obs obeservation

if a cons is public ⇒ Not possible to create it as singlaton

⇒ if a cons is private ⇒ problem problem because private can't access directly

⇒ But private things can be accessed from methods of same class!!!

so but we can access private from a method inside that class so we can create a method inside that same class and call that constructer

```
Class Database {
    ≡≡≡

    private Database() { }

    public Database getInstance() {
        return new Database();
    }
}
```

⇒ But how will you even access the method ∵ to access method you need obj of that class

```
Class Database {

    private Database() { }

    public static Database getInstance() {
        return new Database();
    }
}
```

again, after making static that method again someone can create multiple objects as below so to handle it define a static variable and check if there not any instance then creates

```
Database db1 = Database.getInstance();
db2 =
```

2 obj.

```
Class Database {

    private static Database instance = null;

    private Database() { }

    public static Database getInstance() {
        if (instance == null) {
            instance = new Database();
        }
        return instance;
    }
}
```

Solution 1

```
Class Database {
    private static Database instance = null;
    private String url;
    private String username;
    private String password;
    private Database(){ //initiate conn pool }

    public static Database getInstance(){
        if (instance == null){
            instance = new Database();
        }
        return instance;
    }

    public void setUrl(url){
        this.url = url
    }
}
```

T1 → public static Database getInstance(){
T2 → public static Database getInstance(){
T1 → if (instance == null){
T2 → if (instance == null){
T1 → instance = new Database();
T2 → instance = new Database();

but not good for multithread process

Now assume we are in a multithreaded env;

T1	T2
Database.getInstance()	Database.getInstance()

→ 2 obj get created!!

Solution 2 : Eager Initialization
→ avoids conc problem

```
class Database {
    private static Database instance = new Database();
    private String url;
    private String username;
    private String password;
    private Database() { //initiate conn pool }
    public static Database getInstance() {
        return instance;
    }
    public void setUrl(url) {
```

Created at class load even before any thread running because static

T1	T2
Database.getInstance()	Database.getInstance()

Pros : Handled conc problem.

Cons: → If obj creation needs things that are only at runtime → Not possible
→ App^n startup will become slow

Class Database {
    private static Database instance = new Database(
        url → mysql://abc.amazonaws.com:1234,
        username, password
    );

    private Database(url, username, pass) {
        =
    }
}

Class load → Before first line of code runs.

Summary

why Singleton
1. Shared resource
2. expensive obj creation
3. no alter } immutable

How to create
1. Private con
   static attr, method.
2. Eager initialization
   ↳ class load time

problems
- Concurrency issue

problems
- not always good
- slow startup.

version-3 → V3 → locks

version-4 of singalten → V4 → Double Check locking

Dagger → Spring Boot by Google.